AWS ECS (Fargate) Deployment Guide

From code to a running app behind a load balancer, step by step

Big picture

```
Your code -> Docker image -> ECR (image storage)
                                    |
     ECS Cluster -> Task Definition -> Service -> Load Balancer -> Internet
```

One-time setup: AWS account, IAM user, VPC, subnets, internet gateway, route tables.

Per application: ECR repository, task definition, ECS service, target group.

Step 0: Prerequisites

- An AWS account (do not use the root user for daily work)
- AWS CLI installed
- Docker installed
- Your app with a working `Dockerfile`

Step 1: IAM user and CLI setup (one time)

1. Go to **IAM > Users > Create user**.
2. Attach the permissions you need. For learning, use `AmazonEC2ContainerRegistryFullAccess`, `AmazonECS_FullAccess` and `IAMReadOnlyAccess`. In production, give only the minimum.
3. Open the user, then **Security credentials > Create access key > CLI**. Save the Access Key and Secret Key. The secret is shown only once.
4. On your terminal, run `aws configure` and enter the values below.

```
aws configure

AWS Access Key ID:      YOUR_ACCESS_KEY
AWS Secret Access Key:  YOUR_SECRET_KEY
Default region name:    ap-south-1
Default output format:  json
```

Verify the setup:

```
aws configure list
aws sts get-caller-identity     # shows your account ID and user
```

> **Never commit access keys to GitHub.**

Step 2: Networking (one time)

Create the VPC

1. Go to **VPC > Create VPC > "VPC and more"**.

2. Choose 2 Availability Zones, 2 public subnets, and 0 or 2 private subnets.
3. Choose NAT gateway "None" for learning, because NAT costs money.

AWS then auto-creates the subnets, internet gateway and route tables. A default VPC also already exists in every region, and you can use that for practice.

Create two security groups

Separate groups for the load balancer and the service are cleaner and safer:

Security group	Inbound rule	Why
`alb-sg`	HTTP 80 (and HTTPS 443) from `0.0.0.0/0`	Public traffic reaches the load balancer
`ecs-service-sg`	Your container port (e.g. 3000) from source = `alb-sg`	Only the load balancer can reach containers

Outbound can stay as the default (all traffic).

Step 3: ECR: push your image

1. Go to **ECR > Create repository** (e.g. `my-app`).
2. Open the repository and click **View push commands**. They look like this:

```
# 1. Login to ECR
aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com

# 2. Build the image
docker build -t my-app .
# On Apple Silicon (M1/M2/M3) use:
# docker build --platform linux/amd64 -t my-app .

# 3. Tag it
docker tag my-app:latest \
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest

# 4. Push it
docker push <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest
```

Step 4: IAM roles for ECS

ECS uses two different roles. People often confuse them.

Role	Used by	Purpose	Policy
Task Execution Role (required)	ECS itself	Pull the image from ECR and send logs to CloudWatch	`AmazonECSTaskExecutionRolePolicy`
Task Role (optional)	Your app code inside the container	Access AWS services like S3, DynamoDB, SQS	Custom policy with only what the app needs

To create each role: **IAM > Roles > Create role > AWS service > Elastic Container Service Task**. This sets the trust relationship to `ecs-tasks.amazonaws.com`. Name the first one `ecsTaskExecutionRole`.

Step 5: ECS cluster and task definition

Cluster

1. Go to **ECS > Clusters > Create cluster**.
2. Name it, and choose **AWS Fargate (serverless)**.

Task definition

1. Go to **ECS > Task definitions > Create new task definition**.
2. Fill in the fields listed below.
3. Click **Create**.

Field	Value
Launch type	AWS Fargate
CPU / Memory	e.g. 0.5 vCPU / 1 GB
Task execution role	`ecsTaskExecutionRole`
Task role	Your app role (or none)
Container name	`my-app`
Image URI	`<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest`
Container port	The port your app listens on (e.g. 3000)
Environment variables	If needed (use Secrets Manager for secrets)
Logging	Keep `awslogs` enabled so you can debug

Step 6: Load balancer and target group

Create these before the service. It is easier than doing it inside the service wizard.

1. Target group

- Go to **EC2 > Target Groups > Create**.
- Target type: **IP addresses** (required for Fargate).
- Protocol/Port: HTTP, your container port. VPC: your VPC.
- **Health check path:** e.g. `/` or `/health`. The path must return HTTP 200.
- Do not register any targets manually. ECS does this.

2. Application Load Balancer

- Go to **EC2 > Load Balancers > Create > Application Load Balancer**.
- Scheme: internet-facing.
- VPC: your VPC, with at least 2 public subnets in different AZs.
- Security group: `alb-sg`.
- Listener: HTTP 80, forward to the target group you created.

Step 7: ECS service

1. Open **ECS > Cluster > Services > Create**.
2. Fill in the fields listed below.
3. Click **Create**.

Field	Value
Compute	Launch type > Fargate
Task definition	The one you created (latest revision)
Desired tasks	1 or 2
Networking	Your VPC and public subnets
Security group	`ecs-service-sg`. The default SG works only if its inbound rules allow traffic from the ALB.
Public IP	**ON** if you use public subnets without NAT. Otherwise the task cannot pull the image from ECR.
Load balancing	Choose the existing ALB, container `my-app:port`, listener 80, and the existing target group.

Step 8: Test

1. Wait until the service shows **Running** and the target group shows **Healthy**.
2. Open **EC2 > Load Balancers**, copy the **DNS name**, and open `http://<alb-dns-name>` in your browser.

Common problems

Problem	Likely cause
Task stops with `CannotPullContainerError`	No public IP or NAT, wrong image URI, or the execution role is missing
Targets are **Unhealthy**	Wrong health check path or port, or `ecs-service-sg` does not allow traffic from `alb-sg`
`exec format error`	Image built on ARM (Mac M-series) but the task runs on x86. Rebuild with `--platform linux/amd64`
502 / 503 from the ALB	The container crashed or the port does not match. Check CloudWatch Logs
App cannot reach S3 and similar	The Task Role is missing or lacks permissions

For logs, go to **ECS > Service > Tasks > Logs tab**, or **CloudWatch > Log groups**.

Cleanup (to avoid charges)

Delete in this order: **ECS service > ECS cluster > load balancer > target group > ECR images (optional) > NAT gateway or Elastic IPs** (if you created any).

> The ALB and NAT gateway are the main costs. Delete them when you finish practising.